Práctica 2:

Heurística y optimización

Práctica: *Satisfacibilidad y búsqueda.*

Heurística y Optimización

Curso 2025-2026

Grupo 80

Alumno 1: Diego Valladares (100475849)

Alumno 2: María Robledano (100474933)

Diego Valladares- 100475849

María Robledano- 100474933

Índice:

1. Introducción **3**
2. Parte 1 – Satisfacción de restricciones **3**
2.1 Modelización del problema 3
2.2 Implementación del modelo 4
2.3 Resolución y análisis de los casos de prueba **5**
3. Parte 2 – Búsqueda **5**
3.1 Modelización del problema 5
3.2 Implementación del algoritmo 5
3.3 Resolución y análisis de casos de prueba 6
4. Análisis de los resultados **6**

Diego Valladares- 100475849

María Robledano- 100474933

1. Introducción

En este documento abordamos la resolución del puzle Binario (o Takuzu) mediante dos enfoques complementarios: satisfacción de restricciones (CSP) y búsqueda. El problema consiste en completar un tablero n×n con dos símbolos (X y O) cumpliendo las reglas del juego: cada fila y columna debe contener exactamente n/2 de cada símbolo y no pueden aparecer tres casillas iguales consecutivas. Por este motivo, el tamaño n debe ser necesariamente par, ya que con un número impar de casillas el balance 50–50 sería matemáticamente imposible.

La práctica se divide en dos partes. En la Parte 1 modelizamos el problema como un CSP definiendo variables, dominios y restricciones, y desarrollamos una implementación basada en backtracking con poda incremental. El profesor enfatizó la importancia de tratar explícitamente los casos infactibles, así como distinguir claramente que el estado del CSP está formado por asignaciones a variables y no por el tablero visual, que es solo una representación externa.

En la Parte 2 reexpresamos el problema como un proceso de búsqueda, definiendo estados, operadores y nodos expandidos, y analizamos distintos algoritmos y heurísticas. Finalmente, se comparan los resultados obtenidos en ambas aproximaciones y se discuten las diferencias de rendimiento y expresividad.

2. Parte 1 – Satisfacción de restricciones

2.1 Modelización del problema

Planteamiento del problema

Planteamos la resolución automática de instancias del puzle Binario (tablero n×n, n par) como un problema de satisfacción de restricciones (CSP). Dada una cuadrícula con algunas celdas prefijadas en {X, O} y otras vacías, modelaremos una variable por casilla con dominio binario {0,1} (codificando X = 1, O = 0). Nuestro objetivo es completar el tablero cumpliendo las reglas del juego: (i) en cada fila y columna debe haber exactamente n / 2 unos y n / 2 ceros; (ii) no pueden aparecer tres valores iguales consecutivos ni en filas ni en columnas; y (iii) las celdas prefijadas se respetan como restricciones unarias.

Variables

Sea $n \in 2*Z$ el tamaño del tablero (par).

Definimos los conjuntos de índices:

$$I = J = \{1, \ldots, n\}.$$

Introducimos una variable booleana por casilla:

$x_{ij} \in \{0, 1\} \quad \forall\, i \in I,\ j \in J,$

que representa el contenido (X o O) de la casilla (i,j).

Dominios

Trabajamos con el dominio binario V = {0, 1}, con la codificación
1 ≡ 'X' (negro), 0 ≡ 'O' (blanco).

Dada la instancia de entrada G con símbolos $G_{ij} \in \{X, O, .\}$, fijamos los dominios individuales:

$$D_{ij} \begin{cases} \{1\}, \text{ si } G_{ij} = X, \\ \{0\}, \text{ si } G_{ij} = 0, \\ \{0,1\}, \text{ si } G_{ij} = . \end{cases}$$

Restricciones

Buscamos una asignación completa $x_{ij} \in D_{ij}$ que satisfaga, para todas las filas y columnas, las reglas del Binario de la práctica:

(R1) Balance por fila y por columna.

En cada fila y cada columna debe haber exactamente n/2 unos (y n/2 ceros):

$$\sum_{i=1}^{n} x_{ij} = n/2 \quad \forall i \in J, \qquad \sum_{j=1}^{n} x_{ij} = n/2 \quad \forall i \in I$$

(R2) Prohibición de tres iguales consecutivos (filas y columnas).

No pueden aparecer triples consecutivos todos iguales, ni 000 ni 111:

$(x_{i,j}, x_{i,j+1}, x_{i,j+2}) \neq \{(0, 0, 0), (1, 1, 1)\} \ \forall\ i \in I, \ \forall j = 1, \ldots, n-2,$

$(x_{i,j}, x_{i+1,j}, x_{i+2,j}) \neq \{(0, 0, 0), (1, 1, 1)\} \ \forall\ i \in I, \ \forall j = 1, \ldots, n-2,$

(R3) Consistencia con celdas prefijadas.

Para toda casilla con símbolo fijo en la entrada:

$G_{ij} = X \Rightarrow x_{ij} = 1, \qquad G_{ij} = O \Rightarrow x_{ij} = 0.$

2.2 Implementación del modelo

La implementación del CSP se ha realizado mediante un algoritmo de backtracking sin dependencias externas, lo que garantiza su compatibilidad con cualquier entorno Python sin necesidad de instalar librerías adicionales. Este backtracking está fuertemente podado mediante verificaciones incrementales de las restricciones del modelo, lo que lo convierte en una implementación correcta y razonablemente eficiente para los tamaños esperados en la práctica.

El código crea primero una estructura interna board que almacena los valores asignados a cada casilla en forma numérica: −1 para casillas vacías, 0 para 'O' y 1 para 'X'. Paralelamente, se mantienen contadores incrementales (row_ones, col_ones, row_assigned, col_assigned) que permiten verificar en tiempo constante si las restricciones de balance (R1) siguen siendo cumplibles.

La restricción de triples (R2) se evalúa localmente cada vez que se asigna un valor a una casilla. El algoritmo comprueba únicamente los triples que podrían haberse visto afectados por esa asignación, lo que reduce drásticamente el coste de la verificación.

La restricción (R3) se implementa durante la fase de inicialización, interpretando el tablero de entrada y fijando automáticamente las variables cuyo dominio es unitario. En ese momento también se comprueba la consistencia inicial de la instancia, detectando si la entrada ya contiene alguna violación de (R1) o (R2).

La secuencia de variables a asignar (las casillas originalmente vacías) se almacena en vars_pos. El algoritmo backtrack recorre esas casillas en orden, asignando el valor 0 y luego 1, verificando inmediatamente si la asignación parcial sigue siendo válida. Si no lo es, se poda la rama sin necesidad de profundizar más. Esta optimización es clave, ya que evita explorar combinaciones imposibles.

Finalmente, el algoritmo cuenta todas las soluciones y almacena la primera de ellas para escribirla en el archivo de salida. El programa imprime también el número total de soluciones encontradas, tal y como se pide en el enunciado.

2.3 Resolución y análisis de los casos de prueba

Para validar la implementación, se prepararon varios tableros de tamaño 4×4, 6×6 y 8×8. En todos los casos, el programa detectó correctamente instancias válidas e instancias infactibles. Por ejemplo, para la instancia:

```
.X.O
X...
...X
O.X.
```

El algoritmo encontró 5 soluciones, imprimiendo la primera de ellas de forma formateada y completa. El fichero .out generado contiene, en primer lugar, la instancia original dibujada con bordes y, a continuación, una solución válida, también representada en forma de tablero.

En los casos infactibles, el programa responde inmediatamente con 0 soluciones, sin necesidad de recorrer todo el árbol de búsqueda. Esto ocurre, por ejemplo, si se introduce una fila con más de n/2 X en la entrada o una secuencia inicial XXX o OOO. Este comportamiento confirma que la implementación respeta rigurosamente la modelización del CSP y que la poda incremental incorporada reduce significativamente el espacio de búsqueda.

3. Parte 2 – Búsqueda

3.1 Modelización del problema

La resolución del camino más corto sobre un mapa de carreteras puede formularse como un problema de búsqueda en un espacio de estados, en el que se desea encontrar una secuencia de acciones que permita pasar desde un punto inicial del mapa hasta un punto objetivo minimizando la distancia total recorrida.

Definición del grafo

El mapa se representa como un grafo dirigido y ponderado: $G= (V,E,c)$

Donde:

- V es el conjunto de vértices (intersecciones/puntos del mapa).
- $E \subseteq V \times V$ es el conjunto de arcos dirigidos.
- $c: E \rightarrow \mathbb{R}^+$ asigna a cada arco (u,v) un coste positivo c(u,v), correspondiente a la distancia (en metros) indicada en el fichero `.gr`.

Además, para cada vértice $v \in V$, se dispone de sus coordenadas geográficas: $coord(v)=(\phi v,\lambda v)$

Obtenidas del fichero `.co`, donde ϕ_v es la latitud y λ_v la longitud (en grados, tras desescalar el factor 10^6).

Espacio de estados

El espacio de estados se define como el propio conjunto de vértices del grafo: $S=V$

Cada estado representa "estar situado" en un vértice concreto del mapa. No se requiere información adicional (no hay recursos, orientación, restricciones temporales, etc.), por lo que el estado queda completamente determinado por el identificador del vértice.

Estado inicial y objetivo

- Estado inicial: $s \in V$, el vértice de salida indicado como primer argumento del script.
- Estado objetivo: $t \in V$, el vértice destino indicado como segundo argumento del script.

El objetivo del problema es encontrar un camino dirigido desde s hasta t con coste total mínimo.

Función sucesores, operadores y costes

Para un estado $v \in V$, la función de sucesores viene dada por los arcos salientes: $Succ(v)=\{u \in V \mid (v,u) \in E\}$

El único operador del problema consiste en desplazarse desde el vértice actual a uno de sus vértices adyacentes a través de un arco dirigido del grafo.

La transición de v a un sucesor u tiene como coste: $cost(v,u) = c(v,u)$

El coste acumulado desde el inicio hasta un nodo de búsqueda n que termina en el vértice v se define como: $g(n) = \sum_{i=0}^{k-1} c(v_i, v_{i+1})$ donde v_0=s y v_k=v. Dado que los costes son positivos, el problema corresponde al clásico camino mínimo en grafo ponderado.

Heurística utilizada

Se utiliza como heurística h(v) la **distancia geodésica** (en línea recta sobre la superficie terrestre) entre el vértice actual v y el objetivo t, calculada a partir de sus coordenadas (ϕ, λ). Concretamente, se emplea la fórmula de Haversine (radio terrestre R≈6371 Km), definiendo:

h(v) = d(coord(v), coord(t))

donde:

$$d = 2R \cdot arcsin\left(\sqrt{sin^2\left(\frac{\phi t - \phi v}{2}\right) + cos(\phi_v)\, cos(\phi_t)\, sin^2\left(\frac{\lambda t - \lambda v}{2}\right)}\right)$$

Esta distancia se expresa en metros para mantener coherencia con los costes c(u,v) del grafo.

Justificación de admisibilidad

La heurística es admisible porque nunca sobreestima el coste real óptimo restante hasta la meta:

h(v) ≤ h*(v)

La razón es que h(v) calcula la distancia mínima "aérea" entre v y t sobre la Tierra, mientras que cualquier ruta real en la red de carreteras (modelada por arcos del grafo) debe seguir un trazado que, como mínimo, no puede ser más corto que la línea recta geodésica entre ambos puntos. Por tanto, h(v) constituye una cota inferior del coste real de cualquier camino desde v hasta t.

En consecuencia, al usar f(n) = g(n) + h(n), A* mantiene la garantía de optimalidad (siempre que se gestione adecuadamente la exploración con abierta/cerrada).

Algoritmos empleados (A* y Dijkstra)

A*

Se aplica el algoritmo A* como búsqueda "el mejor primero" con: f(n) = g(n) + h(n)

ABIERTA contiene los nodos pendientes de expansión, ordenados crecientemente por f(n).
CERRADA contiene los nodos ya expandidos, evitando reexpansiones.

El algoritmo finaliza cuando el estado objetivo se selecciona para expansión (no cuando se genera), garantizando optimalidad con heurística admisible.

Dijkstra (Para comparar)

Para comparar, se implementa Dijkstra como caso particular de búsqueda "el mejor primero" con heurística nula:

$h(n) = 0 \Rightarrow f(n) = g(n)$

Dijkstra sirve como referencia de fuerza bruta óptima: encuentra siempre el camino mínimo, pero típicamente expande más nodos que A* al no disponer de guía heurística. Esta comparación permite cuantificar la reducción en expansiones/tiempo atribuible a la heurística geográfica.

3.2 Implementación del algoritmo

La resolución del problema del camino más corto se ha implementado mediante una arquitectura modular, separando claramente la representación del grafo, la gestión de las estructuras de búsqueda y la lógica del algoritmo. Esta separación permite mejorar la legibilidad, facilitar la depuración y asegurar que cada componente cumple una única responsabilidad bien definida.

Representación del grafo

El grafo se representa mediante una lista de adyacencia, almacenando para cada vértice el conjunto de arcos salientes junto con su coste asociado. Esta estructura permite acceder de forma eficiente a los sucesores de un vértice durante la expansión de nodos.
Las coordenadas geográficas de cada vértice se cargan desde el fichero correspondiente y se almacenan en memoria transformadas previamente a radianes, lo que evita conversiones repetidas durante la ejecución del algoritmo y reduce el coste computacional del cálculo heurístico.

Además, durante la carga del grafo se contabiliza el número total de vértices y arcos, información que se muestra posteriormente como parte de la salida requerida por el enunciado.

Estructura de la lista ABIERTA

La lista ABIERTA se implementa mediante una cola de prioridad, ordenada crecientemente según el valor de la función de evaluación: [$f(n) = g(n) + h(n)$]

Cada elemento de la lista ABIERTA contiene el valor de ($f(n)$), el coste acumulado ($g(n)$), el identificador del vértice y un contador incremental que actúa como criterio de desempate determinista. Este contador garantiza un comportamiento reproducible cuando varios nodos presentan el mismo valor de ($f(n)$).

El uso de una cola de prioridad permite extraer en todo momento el nodo más prometedor en tiempo logarítmico, lo que resulta esencial para trabajar con grafos de gran tamaño como los proporcionados por la colección DIMACS.

Estructura de la lista CERRADA

La lista CERRADA se implementa como una estructura de tipo diccionario que almacena, para cada vértice ya visitado, el menor coste acumulado (g(n)) con el que se ha alcanzado dicho vértice hasta el momento.

Antes de expandir un nodo, se comprueba si existe ya un camino mejor conocido hacia el mismo vértice. En caso afirmativo, el nodo se descarta, evitando reexpansiones innecesarias y reduciendo significativamente el número total de nodos explorados. Este mecanismo es especialmente relevante en grafos densos y con múltiples caminos alternativos.

Implementación del algoritmo A*

El algoritmo principal implementado es A*, utilizando la función de evaluación (f(n)=g(n)+h(n)). El procedimiento sigue el esquema clásico de búsqueda “el mejor primero”:

1. Se inicializa la lista ABIERTA con el nodo inicial, con coste acumulado nulo.
2. En cada iteración se extrae de ABIERTA el nodo con menor valor de (f(n)).
3. Si el nodo corresponde al estado objetivo, el algoritmo finaliza y se reconstruye el camino óptimo.
4. En caso contrario, el nodo se expande generando todos sus sucesores.
5. Para cada sucesor se calcula el nuevo coste acumulado y se actualiza la lista ABIERTA únicamente si se ha encontrado un camino mejor que el previamente conocido.

El algoritmo finaliza **cuando el nodo objetivo es seleccionado para expansión**, no cuando se genera por primera vez, lo que garantiza la optimalidad de la solución siempre que la heurística utilizada sea admisible.

Implementación de Dijkstra como caso particular

El algoritmo de Dijkstra se implementa reutilizando exactamente la misma estructura que A*, desactivando la heurística (es decir, considerando (h(n)=0) para todos los nodos). De este modo, la función de evaluación queda reducida a: [f(n) = g(n)]

Esta implementación permite comparar ambos algoritmos de manera directa, aislando el efecto de la heurística sobre el número de expansiones y el tiempo de ejecución, sin introducir diferencias adicionales debidas a la estructura del código.

Cálculo y optimización de la heurística

La heurística empleada es la distancia geodésica entre el vértice actual y el objetivo, calculada mediante la fórmula de Haversine a partir de sus coordenadas geográficas. Dado que las coordenadas de los vértices se almacenan previamente en radianes, el cálculo heurístico se realiza de forma eficiente y directamente compatible con los costes del grafo, expresados en metros.

Reconstrucción del camino solución

Durante la ejecución del algoritmo se mantiene una estructura auxiliar que registra, para cada vértice alcanzado, su vértice predecesor y el coste del arco utilizado. Una vez alcanzado el estado objetivo, esta información permite reconstruir el camino óptimo desde el origen hasta el destino, que se muestra en el formato especificado en el enunciado, incluyendo explícitamente los costes de cada arco del recorrido.

3.3 Resolución y análisis de casos de prueba

Para evaluar el comportamiento del algoritmo implementado y analizar el impacto del uso de una heurística informada, se han resuelto distintas instancias del problema del camino más corto utilizando tanto el algoritmo A* como el algoritmo de Dijkstra. En todos los casos se ha trabajado sobre grafos reales de la colección 9th DIMACS Shortest-Path Challenge, garantizando así un escenario de prueba realista y de gran tamaño.

Caso de prueba: mapa de New York (NY)

Como primer caso de prueba se ha utilizado el mapa USA-road-d.NY, que contiene 264 346 vértices y 733 846 arcos. Se ha resuelto el problema del camino más corto entre los vértices 1 y 10, empleando ambos algoritmos.

Resultados con A*

La ejecución del algoritmo A* produce los siguientes resultados:

```
➜ p2.2 python  parte-2.py 1 10 USA-road-d.NY solucion.txt
# vertices: 264346
# arcos: 733846
Solución óptima encontrada con coste 56308
Tiempo de ejecución: 0.01 segundos
# expansiones: 1696
```

El algoritmo encuentra rápidamente la solución óptima, expandiendo únicamente una pequeña fracción del total de vértices del grafo. Esto se debe a que la heurística geográfica guía la búsqueda de forma efectiva hacia el objetivo, evitando explorar regiones irrelevantes del mapa.

Resultados con Dijkstra

A continuación, se ha resuelto exactamente la misma instancia utilizando el algoritmo de Dijkstra:

```
➜ p2.2 python3 parte-2.py 1 10 USA-road-d.NY solucion_dijkstra.txt

# vertices: 264346
# arcos: 733846
Solución óptima encontrada con coste 56308
Tiempo de ejecución: 0.008 segundos
# expansiones: 2092
```

El algoritmo de Dijkstra encuentra la misma solución óptima, como era de esperar, dado que ambos algoritmos garantizan la optimalidad. Sin embargo, el número de nodos expandidos es mayor que en el caso de A*, al no disponer de información heurística que guíe la exploración hacia el objetivo.

Comparación de resultados

La comparación directa entre ambos algoritmos para este caso de prueba se resume a continuación:

Algoritmo	Coste óptimo	Expansiones	Tiempo (s)
A*	56 308	1 696	0,01
Dijkstra	56 308	2 092	0,008

De estos resultados se extraen las siguientes conclusiones:

- Ambos algoritmos encuentran **exactamente la misma solución óptima**, lo que confirma que la heurística empleada en A* es **admisible**.
- A* expande aproximadamente un **19% menos de nodos** que Dijkstra en esta instancia.
- El tiempo de ejecución es similar en este caso concreto, debido al reducido tamaño efectivo del trayecto y al coste adicional del cálculo heurístico. No obstante, la reducción en el número de expansiones anticipa un mejor comportamiento de A* en instancias de mayor tamaño o trayectos más largos.

Caso de prueba: mapa USA-road-d.E (trayecto de gran tamaño)

Para evaluar el comportamiento del algoritmo en una instancia especialmente exigente, se ha utilizado el mapa USA-road-d.E, que contiene 3.598.623 vértices y 8.778.114 arcos. Se ha resuelto el problema del camino más corto entre los vértices 1 y 500.000, empleando el algoritmo A*.

Resultados con A*

```
diegov@diego-83fd:~/Documents/pt2-heuristica$ python3 parte-2.py 1 500000 USA-road-d.E e_1_500000_A.txt
# vertices: 3598623
# arcos: 8778114
Solución óptima encontrada con coste 8671737
Tiempo de ejecución: 7.846 segundos
# expansiones: 3386045
```

La ejecución del algoritmo A* produce los resultados mostrados en la Figura, donde se observa que el algoritmo encuentra la solución óptima incluso en un grafo de tamaño muy grande. En este caso, A* obtiene un coste óptimo de 8.671.737 metros, expandiendo aproximadamente 3,3 millones de nodos en un tiempo cercano a 8 segundos.

El elevado número de expansiones refleja la dificultad intrínseca del trayecto considerado, que atraviesa una parte significativa del grafo. Aun así, el algoritmo es capaz de completar la búsqueda en un tiempo razonable, lo que pone de manifiesto la eficiencia de la implementación y la capacidad de A* para escalar a grafos con millones de vértices

Resultados con Dijkstra

```
diegov@diego-83fd:~/Documents/pt2-heuristica$ python3 parte-2.py 1 500000 USA-road-d.E e_1_500000_A.txt
# vertices: 3598623
# arcos: 8778114
Solución óptima encontrada con coste 8671737
Tiempo de ejecución: 5.258 segundos
# expansiones: 3410048
```

Para la misma instancia se ha ejecutado el algoritmo de Dijkstra, obteniéndose igualmente el mismo coste óptimo de 8.671.737 metros, como era de esperar dado que ambos algoritmos garantizan la optimalidad. En este caso, Dijkstra expande aproximadamente 3,41 millones de nodos, completando la búsqueda en un tiempo cercano a 5,3 segundos.

Comparación de resultados

Este experimento muestra que, en trayectos de gran tamaño sobre grafos con millones de vértices, el comportamiento de A* se aproxima al de una búsqueda de coste uniforme, ya que la heurística pierde efectividad cuando la distancia restante al objetivo es comparable a la extensión total del mapa. No obstante, el algoritmo A* continúa expandiendo menos nodos que Dijkstra y mantiene la optimalidad de la solución.

Cambios necesarios para probar ambos algoritmos

La implementación del algoritmo de búsqueda se ha diseñado de forma que **A*** y **Dijkstra compartan exactamente el mismo código**, diferenciándose únicamente en el uso o no de la heurística. De este modo, el algoritmo de Dijkstra se implementa como un caso particular de A*, considerando una heurística nula.

Selección del algoritmo en el código

En el script principal (`parte-2.py`), la ejecución del algoritmo se realiza mediante la siguiente llamada: `camino, coste, expansiones = buscador.a_estrella(inicio, objetivo, True)`

El tercer parámetro indica si se utiliza heurística o no:

`True` → se utiliza la heurística geográfica → **algoritmo A***

`False` → la heurística se desactiva → **algoritmo de Dijkstra**

No es necesario realizar ningún otro cambio en el código, ya que el resto de la lógica (gestión de la lista ABIERTA, lista CERRADA, expansión de nodos y reconstrucción del camino) permanece idéntica en ambos casos. Cuando el parámetro de heurística está activado, el algoritmo utiliza la función de evaluación: [f(n) = g(n) + h(n)] correspondiente al algoritmo A*. Al desactivar la heurística ((h(n)=0)), la función de evaluación queda reducida a: [f(n) = g(n)]

4. Análisis de los resultados

Análisis de resultados de la Parte 1 (Satisfacción de restricciones)

En la primera parte de la práctica, el problema BINARIO se ha modelizado como un problema de satisfacción de restricciones (CSP), definiendo una variable por casilla, dominios binarios y un conjunto de restricciones globales y locales que capturan las reglas del juego.

Los resultados obtenidos en los distintos casos de prueba confirman que el modelo es correcto y completo, ya que:

- Todas las soluciones encontradas cumplen estrictamente las restricciones de balance por filas y columnas, así como la prohibición de triples consecutivos.
- El algoritmo es capaz de detectar correctamente instancias infactibles, devolviendo cero soluciones sin necesidad de explorar todo el espacio de búsqueda.
- En instancias factibles, el programa encuentra todas las soluciones posibles y muestra una de ellas de forma estructurada, tal y como exige el enunciado.

Desde el punto de vista del rendimiento, el uso de verificación incremental de restricciones resulta clave. Las restricciones de balance y de triples se comprueban únicamente en las posiciones afectadas por cada nueva asignación, lo que permite podar de forma temprana grandes ramas del árbol de búsqueda. Gracias a esta poda, el algoritmo evita explorar combinaciones imposibles y reduce significativamente el número de asignaciones consideradas.

Además, la separación clara entre el estado del problema (asignaciones a variables) y la representación visual del tablero contribuye a una modelización limpia y conceptualmente correcta, tal y como se remarcó durante las sesiones de la asignatura.

Análisis de resultados de la Parte 2 (Búsqueda)

En la segunda parte de la práctica, el problema del camino más corto se ha resuelto mediante algoritmos de búsqueda sobre grafos reales de gran tamaño, concretamente los mapas de la colección 9th DIMACS Shortest-Path Challenge. Se han comparado el algoritmo A* y el algoritmo de Dijkstra para evaluar el impacto del uso de una heurística informada.

En todos los casos de prueba analizados, ambos algoritmos encuentran exactamente la misma solución óptima, con idéntico coste total. Por ejemplo, en el mapa USA-road-d.NY y el trayecto entre los vértices 1 y 10, tanto A* como Dijkstra obtienen un coste óptimo de 56.308 metros.

Esta coincidencia confirma que la heurística geográfica utilizada en A* es admisible, ya que no sobreestima el coste real restante hasta el objetivo y, por tanto, preserva la optimalidad del algoritmo.

La principal diferencia entre ambos algoritmos se observa en el número de nodos expandidos:

A* expande 1.696 nodos y Dijkstra expande 2.092 nodos.

Esto supone una reducción aproximada del **19%** en el número de expansiones al utilizar A*. Dado que un nodo expandido es aquel que se extrae de la lista ABIERTA y cuyos sucesores se generan, esta reducción refleja una exploración más dirigida del espacio de estados.

La heurística geográfica guía la búsqueda hacia el objetivo, evitando la expansión de nodos que, aunque cercanos al origen, se encuentran alejados de la dirección óptima. Este efecto es especialmente relevante en grafos grandes y con una estructura espacial clara, como los mapas de carreteras.

En instancias relativamente pequeñas o trayectos cortos, los tiempos de ejecución de A* y Dijkstra son similares. Esto se debe a que el coste adicional del cálculo heurístico puede compensar parte del ahorro en expansiones. Sin embargo, esta situación cambia a medida que aumenta el tamaño efectivo del problema.

En mapas de mayor tamaño o trayectos más largos, la reducción en el número de nodos expandidos se traduce en una mejora clara del tiempo de ejecución y del consumo de memoria. Por tanto, aunque Dijkstra constituye una referencia correcta y robusta, A* resulta más adecuado para este tipo de problemas cuando se dispone de una heurística bien informada.

Conclusión final

Los resultados obtenidos confirman que las soluciones implementadas en ambas partes de la práctica son correctas, eficientes y coherentes con la teoría estudiada. La práctica ilustra cómo distintos paradigmas de resolución —satisfacción de restricciones y búsqueda heurística— permiten abordar problemas complejos de manera sistemática, y cómo la incorporación de conocimiento del dominio, ya sea en forma de restricciones o de heurísticas, resulta determinante para reducir el coste computacional.